

반복되던 업무를 직접 만든 도구로 바꿨습니다

AI 도구를 도입한 경험이 아니라, 필요한 도구를 직접 설계하고 만들어 팀에 붙인 기록입니다. 기획 산출물을 만드는 방식과 프로젝트를 관리하는 방식, 두 구간을 바꿨고 지금도 매일 쓰고 있습니다.

영업일 5~7일

→ 30~40분
착수 기획 산출물 초안

2

개
직접 설계·구축해 운영 중인 시스템

20

개 이상
적용한 고객사 프로젝트

0

건
도입에 쓴 외부 솔루션 비용

PRD Toolkit · 기획 산출물 자동 생성

계약서 PDF를 넣으면 기능 정의 → 페이지 맵 → 동작하는 HTML 시안 → 화면별 기획서 · 테스트케이스까지 한 번의 실행으로 나옵니다. 배포와 고객 피드백 수집까지 이어집니다.

PM Station · 프로젝트 관리 · 보고 자동화

마감 있는 모든 일이 한 화면에 모이고, 정해진 시각에 Teams로 먼저 알림이 갑니다. 운영 지표 보고서는 매일 자동으로 만들어져 발송됩니다.

김석남 · AI 서비스 기획 · PM

ramseok.github.io · rlatjrsk1674@gmail.com

본 문서의 모든 화면은 실제 운영 중인 시스템입니다.
고객사명 · 개인정보는 A사 · B사 형태로 치환해 촬영했습니다.

도구를 도입해서 풀리지 않던 두 구간

SI 환경에서 기획과 PM을 겸임하며 여러 프로젝트를 병렬로 운영했습니다. 일이 막히는 지점은 늘 같았습니다. **착수 산출물을 손으로 그리는 구간**과 **진행 상황이 사람 머릿속·메신저에 흩어지는 구간**이었습니다. 새 협업툴을 하나 더 들이는 방식으로는 풀리지 않았습니다. 일하는 순서 자체를 바꿔야 했습니다.

① 기획 산출물 — 매번 처음부터

Before

- 계약서를 읽고 기능을 다시 정리
- Figma로 화면을 한 장씩 작도
- PDF 변환·병합 후 메일 전달
- 피드백은 메신저·메일에 흩어짐



After — PRD Toolkit

- 계약서 PDF를 넣으면 기능 리스트 추출
- 동작하는 HTML 시안까지 자동 생성
- 링크 하나로 전달, 배포까지 자동
- 피드백·테스트케이스를 화면에 붙여 보관

② 프로젝트 관리 — 기억에 의존

Before

- 마감·요청이 메신저와 개인 메모에 분산
- 고객사 회신 대기 건을 놓침
- 운영 지표는 매일 수기로 확인·작성
- 누가 무엇을 바꿨는지 추적 불가



After — PM Station

- 마감 있는 모든 항목이 한 화면에 모임
- 정해진 시각에 Teams로 먼저 알림
- 운영 보고서를 매일 자동 생성
- 변경 이력 자동 기록

이 과정에서 세운 기준

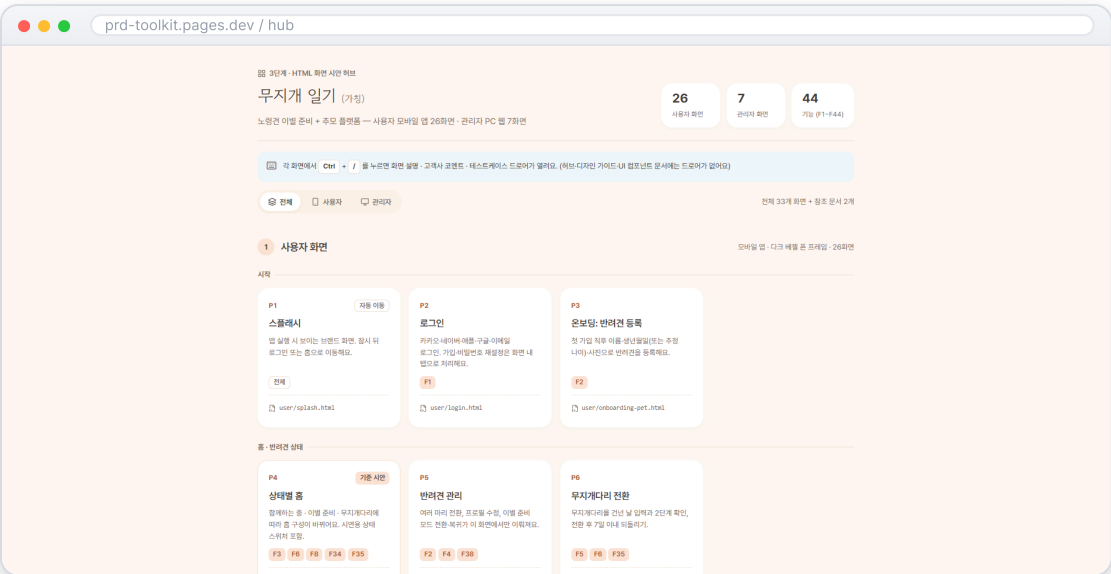
사람이 도구로 가지 않게 한다. 접속해야 보이는 정보는 결국 안 봅니다. 알림도 보고서도 사용자가 있는 자리로 먼저 보냈습니다.

자동화 앞에 게이트를 둔다. 만든 주체가 스스로 통과 판정을 내리지 못하게 하고, 완료는 파일 개수가 아니라 내용 기준으로 검사합니다.

근거 없는 답은 내보내지 않는다. 사내 자료에 근거가 없으면 AI가 답을 만들지 않고 정해진 안내 문구를 반환하도록 규칙을 정의했습니다.

PRD Toolkit — 계약서를 넣으면 기획 산출물이 나옵니다

기획 산출물 자동화



시안 허브 — 한 번의 실행으로 만들어진 전체 산출물이 한 화면에 모입니다. 화면마다 관련 기능 번호(F1~F44)가 붙어 있어 계약 범위와 화면이 1:1로 추적됩니다.

무엇이 달라졌나

- **시작점이 계약서** — 사람이 명세를 다시 쓰는 단계를 없앴습니다. 계약 범위 밖 기능이 기획서에 섞여 들어가는 일도 줄었습니다.
- **초안 영업일 5~7일 → 30~40분** — 이후 고객 협의에 따른 수정은 별도지만, 출발선이 달라졌습니다.
- **움직이는 시안** — 정지된 이미지가 아니라 클릭이 되는 화면이라, 고객이 그 자리에서 판단합니다.

실제 산출 규모 (1건 기준)

사용자 화면	26	관리자 화면	7
정의 기능	44 (F1~F44)	참조 문서	2
화면별 기획서	33	테스트케이스	화면당 17~20

* 규모는 계약 범위에 따라 달라집니다. 위 수치는 예시 1건 기준입니다.

Claude Code

에이전트 9종 분업

품질 게이트

Supabase

Netlify · Vercel 자동 배포

이 화면에서 바로 보이는 것

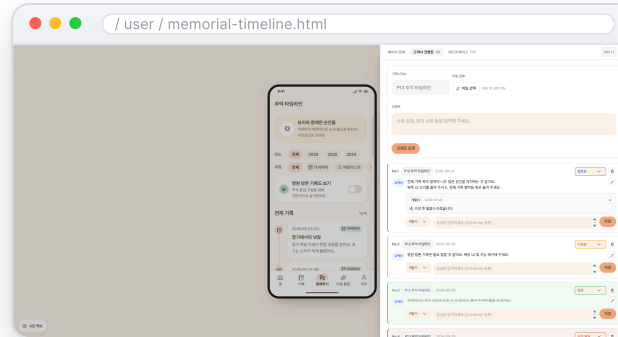
상단에 **사용자 26 · 관리자 7 · 기능 44**가 집계되고, 화면 카드마다 담당 기능 태그와 파일 경로가 붙습니다. 고객은 목록을 훑으며 빠진 화면을 바로 짚을 수 있고, 개발팀은 파일 단위로 착수할 수 있습니다.

시안 위에서 기획서 · 피드백 · 테스트를 함께 봅니다

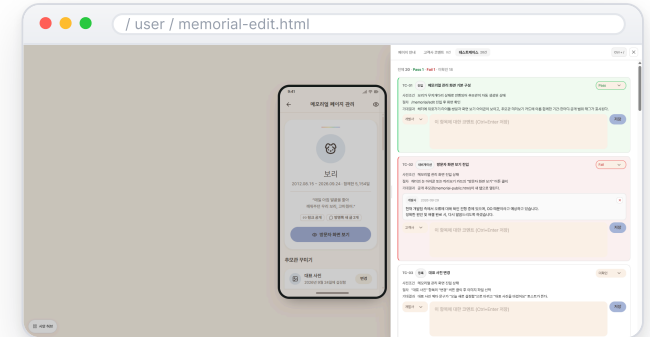
산출물이 빨리 나오는 것보다 중요한 것은 **흠어지지 않는 것**이었습니다. 기획서는 문서함에, 피드백은 메신저에, 테스트는 엑셀에 있으면 어떤 피드백이 어느 화면의 어느 항목인지 매번 다시 맞춰야 합니다. 그래서 시안 화면에서 **Ctrl + /** 를 누르면 우측 패널이 열리고, 그 화면의 기획서 · 고객사 코멘트 · 테스트케이스가 한자리에서 열리도록 했습니다.



① **화면별 기획서** — URL · 대상 · 관련 기능이 머리에 붙고, 화면 구성이 영역 단위 표로 정리됩니다. 개발팀이 바로 읽는 문서입니다.



② **고객사 코멘트** — 고객이 화면을 보며 남긴 요청이 **검토중 · 수정중 · 완료 · 논의 필요** 상태로 관리되고, 답변 이력이 함께 남습니다.



③ **테스트케이스** — 사전조건 · 절차 · 기대결과가 화면 단위로 붙고 Pass / Fail 로 관리됩니다. QA 결과가 기획서 옆에 남습니다.

여기서 얻은 것

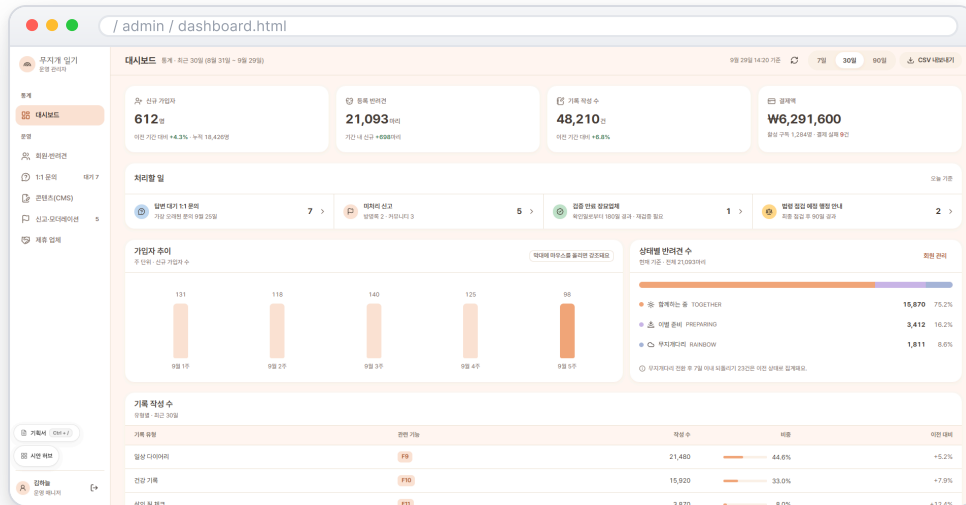
- **재확인 커뮤니케이션 제거** — "그거 어느 화면 얘기였죠?" 를 다시 묻지 않게 됐습니다.
- **합의 이력 보존** — 고객 요청과 화신이 화면에 붙어 남아, 범위 다툼의 근거가 됩니다.

기술적으로 직접 다룬 것

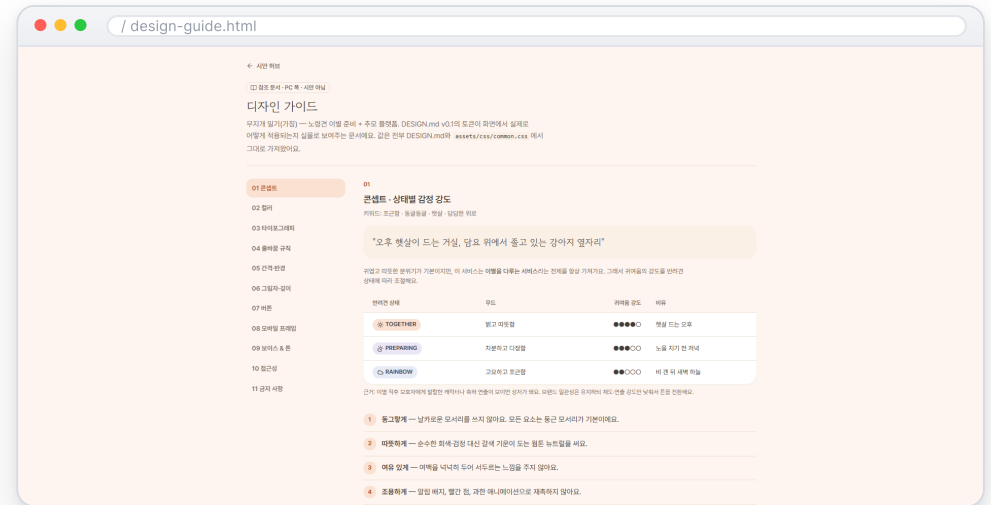
피드백 저장을 위한 **DB 연동(Supabase)**, 프로젝트별 데이터 분리, **정적 호스팅 자동 배포**, 문서 상태 동기화. 기획 직인이지만 별도 개발 리소스 없이 만들고 운영합니다.

관리자 화면과 디자인 기준까지 같은 실행에서 나옵니다

산출물 표준화



관리자 화면 시안 — 지표 카드 · 처리할 일 · 추이 그래프 · 분포까지 구성해 운영자가 무엇을 보고 무엇을 처리하는지 화면으로 합의합니다. (표시 데이터는 예시값)



디자인 가이드 — 고객사 디자인 요구사항과 레퍼런스를 토대로 정리해 컬러 · 타이포 · 간격 · 버튼 규칙을 문서로 고정합니다. 주관적 피드백으로 인한 재작업을 줄이는 장치입니다.

품질을 지키는 두 겹

- 검토 단계 분리 — 산출물을 만든 주체가 스스로 통과 판정을 내리지 못하게 했습니다.
- 내용 기반 완료 판정 — 파일이 생성됐는지 아니라 최소 분량과 필수 항목이 채워졌는지로 검사해, 빈 껍데기 산출물이 넘어가지 않습니다.
- 상위 변경 시 하위 무효화 — 기능 정의가 바뀌면 그 아래 화면 기획의 검토 상태가 자동으로 풀립니다.

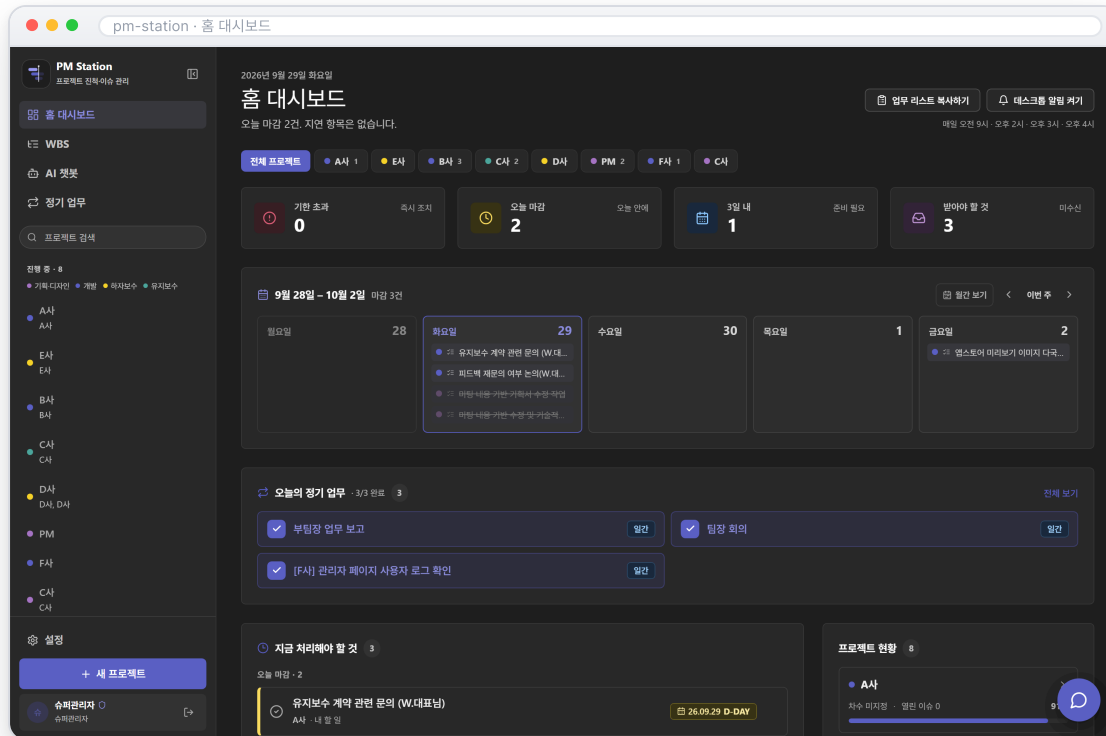
모델을 일에 맞게 나눠 썼습니다

단계	성격	배치
계약 해석 · 품질 검토	판단이 무거움	상위 모델
화면 양산	반복 생성	중간 모델
규칙 검사	기계적 확인	경량 모델

비용과 속도를 함께 맞추기 위한 배치입니다. 에이전트 9종이 단계별로 분업합니다.

PM Station — 마감이 있는 모든 일이 한 화면에 모입니다

프로젝트 관리 자동화



홈 대시보드 — 기한 초과 · 오늘 마감 · 3일 내 · 받아야 할 것 네 갈래로 나눕니다. 마지막 항목이 핵심입니다. 내가 할 일이 아니라 **고객** 사 **회신**과 **자료**를 기다리는 건을 따로 세워 두지 않으면 반드시 새기 때문입니다.

왜 만들었나

스타트업 초기에 합류했을 때 업무 시스템이 없었습니다. 먼저 반복 업무를 전부 매뉴얼로 만들어 문서화했는데, **만드는 것보다 찾아 읽게 만드는 것이 훨씬 어려웠습니다.** 그래서 문서를 쌓는 방향이 아니라 **물어보면 답이 나오는 방향**으로 틀었고, 그 끝에 이 시스템이 나왔습니다.

진화 순서

- 1단계 반복 업무 매뉴얼화 → 워드에서 엑셀 시트로 통합
- 2단계 매뉴얼을 학습 데이터로 올려 **질문·답변** 형태로 전환
- 3단계 Claude Code로 **업무 시스템** 구축 — 할 일 · 이슈 · 마감 통합
- 4단계 Teams 알림 · 운영 보고서 자동 발행으로 **사람에게 먼저 도착**

React · Vite

Supabase

pg_cron

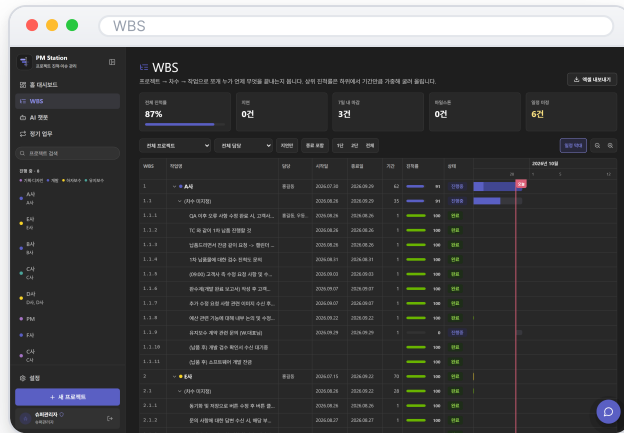
Vercel 서버리스

Teams 연동

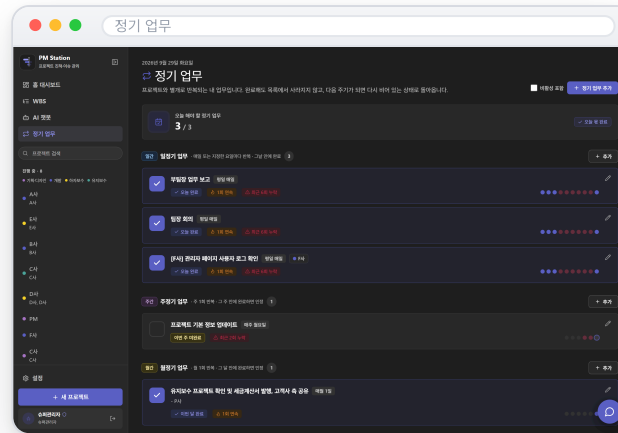
OpenAI

일정 · 정기 업무 · 변경 이력을 자동으로 붙잡습니다

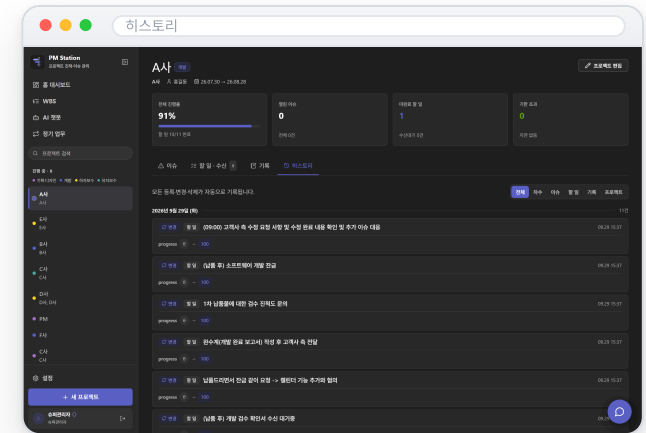
핵심 기능



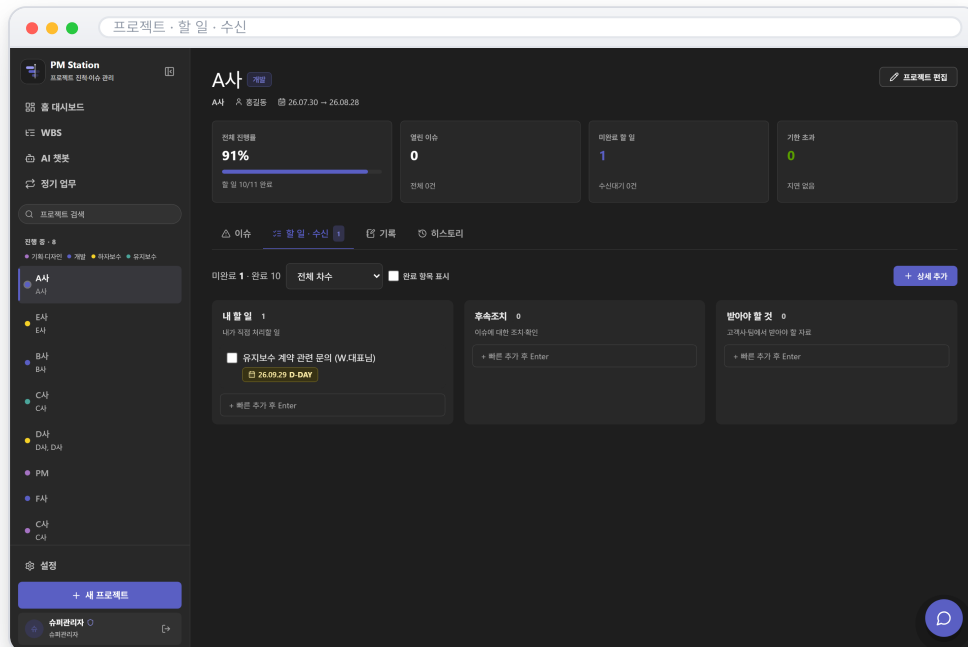
WBS · 간트 — 여러 프로젝트의 단계 일정을 한 줄로 겹쳐 봅니다. 어느 프로젝트가 어느 단계에 있는지, 다음 주에 무엇이 겹치는지 먼저 보입니다.



정기 업무 — 일 · 주 · 월 주기 업무를 체크하고 연속 달성과 누락을 추적합니다. 성실함을 의지로 버티지 않고 **빠뜨리면 시스템이 알려주는** 구조로 바꿨습니다.



변경 이력 — DB 트리거로 누가 무엇을 언제 바꿨는지 자동 기록됩니다. "그때 일정이 왜 바뀌었죠?" 에 근거로 답할 수 있습니다.



프로젝트 상세 — 핵심 행를 · 열린 이슈 · 미완료 할 일 · 기한 초과를 프로젝트 단위로 보고, 이슈 · 할 일 · 기록 · 히스토리 탭으로 나눠 관리합니다. **받아야 할 것**은 별도 레인으로 분리했습니다.

승인제 계정과 권한

가입 신청을 관리자가 승인해야 들어올 수 있고, **담당 프로젝트만** 열람하도록 범위를 나눕니다. 임시 비밀번호 발급 · 최초 로그인 시 변경 강제까지 운영 절차로 넣었습니다.

AI 챗봇 — 사내 문서에 근거해서만 답합니다

등록한 매뉴얼과 PM Station 데이터 안에서 답하고, **근거를 찾지 못하면 모른다고 답합니다**. 답변마다 어떤 문서를 근거로 삼았는지 함께 표시해, 틀린 답이 조용히 퍼지지 않게 했습니다.

도구가 사람에게 먼저 갑니다

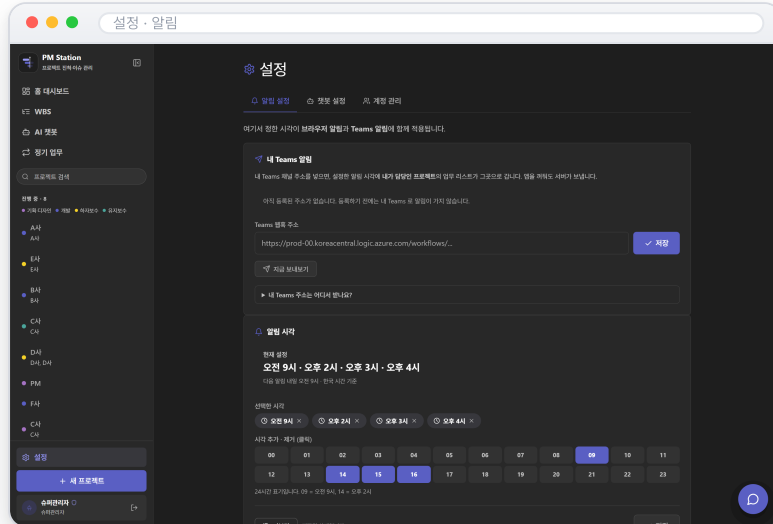
알림 · 자동 보고 · 모바일

운영 보고서 자동 생성

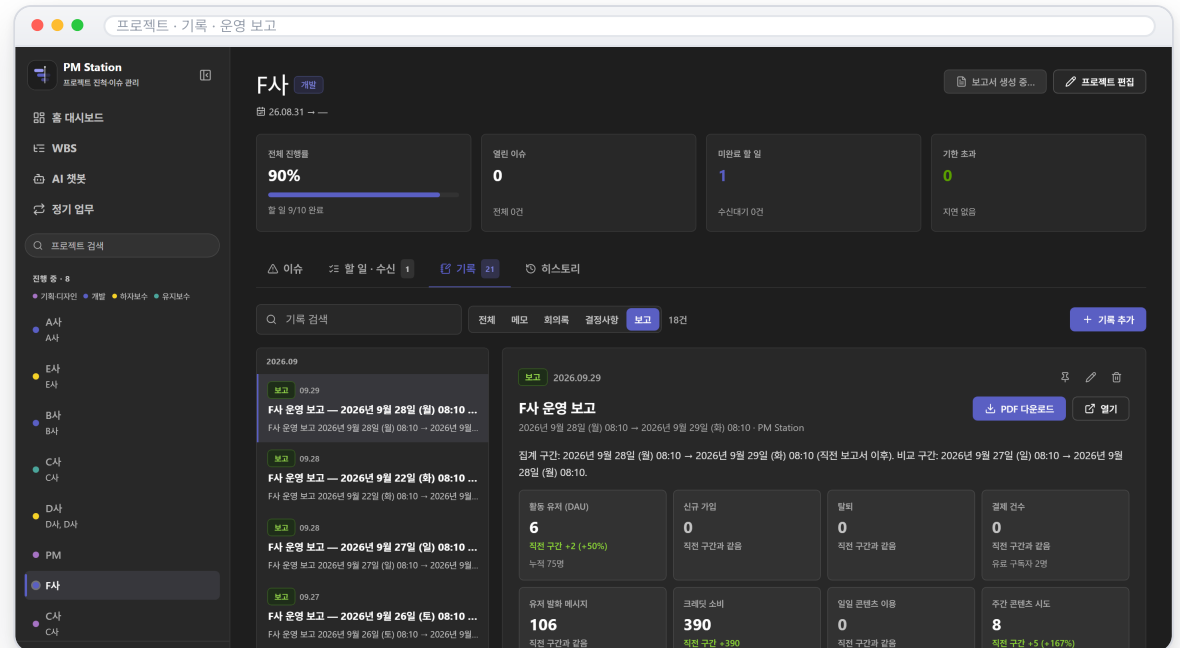
운영 중인 서비스의 관리자 데이터를 읽어 **매일 정해진 시각에 보고서를 만들고 발송합니다.** 활동 유저 · 신규 가입 · 결제 · 크레딧 소비 등 지표와 직전 구간 대비 증감, 조치가 필요한 항목이 함께 정리됩니다. 매일 관리자 페이지를 열어 수기로 작성하던 일이 없어졌습니다.

Teams 알림

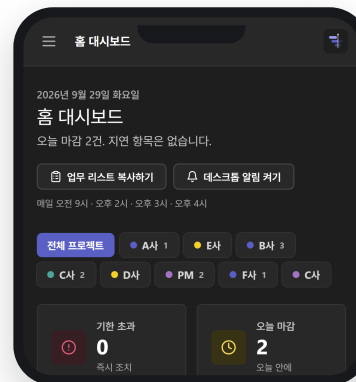
설정한 시각마다 오늘 할 일과 정기 업무를 **개인 채팅으로 발송합니다.** 앱을 열지 않아도 업무가 먼저 도착합니다. 자동화의 성패는 기술이 아니라 **사람이 도구 앞으로 가야 하느냐, 도구가 사람에게 오느냐에서** 갈린다고 생각합니다.



알림 설정 — 발송 시각을 직접 정합니다. 하루 3회가 기본값입니다.



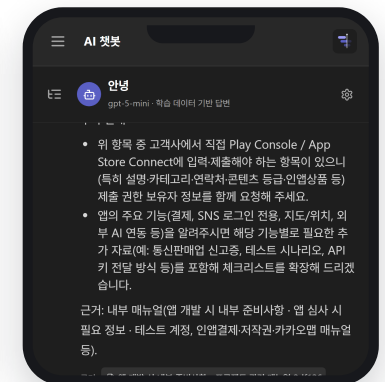
자동 생성된 운영 보고서 — 집계 구간과 비교 구간이 명시되고, 지표별 증감이 함께 표시됩니다. PDF로 내려받아 그대로 보고에 씁니다.



모바일 · 홈
이동 중에도 오늘 마감 확인



모바일 · 정기 업무
현장에서 바로 체크



모바일 · AI 챗봇
필요할 때 바로 질문

일하는 방식을 바꾼 결과와, 다음에 하고 싶은 일

정리

5~7일

→ 30~40분

착수 기획 산출물 초안 소요

4

종 → 1곳

기획서·시안·피드백·TC 통합

매일

자동

수기 운영 보고 → 자동 발행

1일 3회

업무 알림 자동 발송

두 시스템을 만들며 실제로 거친 순서

- 업무를 따라가며 시간이 새는 지점을 찾습니다. 문서와 회의만으로는 안 보여서, 직접 그 자리에 가서 어떤 도구로 어떤 순서로 일하는지 봤습니다.
- 효과와 실행 가능성으로 줄을 세웁니다. 전부 자동화하지 않고, 가장 빨리 체감되는 것부터 손봤습니다.
- 동작하는 형태로 만들어 현업에 붙입니다. 검토 보고서로 끝내지 않고 실제로 쓰는 도구까지 갑니다.
- 적용 전후를 기록해 다음 과제의 근거로 씁니다.

이 경험이 조직에서 쓰이는 방식

- 현업 인터뷰 → 과제 발굴 — 반복 업무와 비효율 구간을 찾아 구조화하는 일을 매 프로젝트마다 했습니다.
- PoC → 배포 → 개선 — 만들어 붙이고 쓰이게 하는 구간까지 직접 해봤습니다.
- 표준화 · 내재화 — 검증된 방식을 문서 표준과 공통 컴포넌트로 남겨 다른 사람이 그대로 쓰게 했습니다.
- 보안 기준 — 외부로 내보낼 수 없는 자료를 다루는 환경에서 AI 사용 범위를 설계한 경험이 있습니다.

한 조직의 도구를 만드는 일에서 멈추지 않고, 회사 전체가 일하는 방식의 기준을 만드는 일을 하고 싶습니다. 먼저 부서를 직접 찾아가 지금 어떤 도구로 어떤 순서로 일하는지 보고, 어디가 불편한지와 무엇을 없앨 수 있는지를 현업과 함께 정리하겠습니다. 그다음 가장 빨리 체감되는 것부터 동작하는 형태로 만들어 붙이고, 적용 전후의 업무시간과 오류를 기록해 다음 과제의 근거로 쓰겠습니다.

김석남

ramseok.github.io · rlatjrka1674@gmail.com · 010-2472-1674

본 문서의 화면은 실제 운영 중인 시스템이며,
고객사명·개인정보는 치환 후 촬영했습니다.